

Plataforma Agenda — Estado del Proyecto

Resumen completo para continuar en otra conversación

Preparado para Jonta

31 de julio de 2026

Índice general

1. Qué es Plataforma Agenda	2
Diferenciadores frente a AgendaPro	2
Stack técnico	2
Convenciones de código	3
2. Todo lo construido hasta ahora, fase por fase	4
Fase 0 — Cimientos	4
Fase 1 — Agenda pública 24/7 + agenda interna	4
Fase 2 — CRM con ficha configurable por vertical	4
Fase 3 — Pagos nativos	4
Fase 4 — Reportes y comisiones	4
Fase 5 — Multi-sede avanzado, CRM predictivo e Inventario	4
Fase 6 — Feature-gating por plan	5
Gestión de profesionales	5
Equipo (invitar usuarios a una sede)	5
Cobro automático del SaaS — Stripe Subscriptions	5
Self-service signup para tenants nuevos	5
Cambio de plan self-service (upgrade/downgrade)	6
Cobro recurrente vía Wompi (Colombia)	6
Recuperación y cambio de contraseña	6
Vínculo Profesional↔Usuario (login propio para un profesional)	6
Gestión de servicios	6
Constructor de campos personalizados por tenant	7
Exportar reportes (CSV, PDF, gráficos)	7
Vínculo servicio↔insumo con descuento automático de stock	7
3. Integración de WhatsApp Business (Meta) — trabajo de esta sesión	8
Lo que se completó	8
Qué falta para que WhatsApp funcione en producción	9
4. Estado de las variables de entorno (.env)	10
5. Qué falta / pendiente para continuar	11
WhatsApp	11

Facturación / planes	11
Seguridad / cuentas	11
Profesionales / equipo	11
Servicios / inventario	12
Reportes	12
General / UX	12
6. Comandos útiles	13
7. Archivos de referencia del proyecto	13

1. Qué es Plataforma Agenda

Una plataforma de agendamiento, CRM y pagos para negocios de salud y bienestar, pensada como alternativa mejor a **AgendaPro**. El nicho inicial de go-to-market es **salud especializada** (psicólogos, nutricionistas, fisioterapeutas, clínicas pequeñas/medianas): el hueco que AgendaPro deja mal servido (fichas clínicas genéricas, sin CIE-10, sin plantillas por especialidad).

Diferenciadores frente a AgendaPro

Estos cinco principios guían cada decisión de producto y **no deben romperse** al construir nuevas funciones:

1. **Precio transparente** — sin add-ons obligatorios para funciones básicas de WhatsApp/recordatorios.
2. **Permisos granulares por sede desde el día 1** (`StaffLocationRole`), no un parche posterior.
3. **Ficha de cliente configurable por vertical** vía `Client.customFields` (JSONB), no un esquema rígido.
4. **Pagos nativos con confirmación en tiempo real** (webhook), nunca con demora de días.
5. **CRM predictivo**, no solo manual: recordatorios automáticos por inactividad de recompra, no solo por cumpleaños.

Stack técnico

- **Next.js (App Router) + TypeScript** — un solo repo para frontend y API.
- **PostgreSQL + Prisma** como ORM (schema-first). Base de datos alojada en Neon.
- **Tailwind CSS** para estilos.
- **Auth.js v5** (next-auth) con `@auth/prisma-adapter` y provider de Credentials (email + contraseña, `bcryptjs`). Sesión JWT con `tenantId` y `StaffLocationRole` embebidos (solo para UI — las mutaciones siempre re-verifican permisos contra la base de datos).
- **Pagos**: Stripe (internacional) + Wompi (Colombia/Bancolombia). El modelo `Payment` contempla ambos proveedores vía el enum `PaymentProvider`.

- **WhatsApp:** WhatsApp Business Cloud API (Meta oficial, sin Twilio ni capa intermedia).

Convenciones de código

- Todo modelo de negocio cuelga de `tenantId` (multi-tenant) — nunca cruzar tenants en una query sin querer.
- Los permisos se validan a nivel de `locationId`, no solo `tenantId`.
- Los campos “flexibles” (ficha clínica, ficha estética, etc.) van en `Json` (`customFields`), nunca como columnas nuevas.
- Nombres de archivos/componentes en inglés, textos de UI en español (mercado LatAm).
- Cada módulo nuevo lleva tests básicos (Vitest para lógica, Playwright para flujos críticos).
- Nunca borrado real de recursos de negocio (sedes, profesionales, servicios, ítems de inventario, campos personalizados) — solo un flag `active`.

2. Todo lo construido hasta ahora, fase por fase

Fase 0 — Cimientos

Schema multi-tenant + RBAC por sede (`StaffLocationRole`). Base de todo lo demás.

Fase 1 — Agenda pública 24/7 + agenda interna

Agenda pública de reservas y agenda interna con login (`Auth.js v5`, `requireDashboardAccess`). Recordatorios automáticos por WhatsApp: al reservar se encola un `NotificationQueue` (canal WHATSAPP) 24h antes de la cita; el cron `/api/cron/send-reminders` (protegido por `CRON_SECRET`) procesa los pendientes vencidos. Archivos clave: `src/lib/whatsapp.ts`, `src/lib/reminderScheduling.ts`.

Fase 2 — CRM con ficha configurable por vertical

`Tenant.vertical` (`GENERAL/PSICOLOGIA/NUTRICION/FISIOTERAPIA/ESTETICA`) con plantillas fijas de campos por vertical (`src/lib/clientFieldTemplates.ts`), guardadas en `Client.customFields`. CRUD de clientes con historial de citas.

Fase 3 — Pagos nativos

Stripe: Checkout hosted + webhook con confirmación en tiempo real. **Wompi (Colombia):** Web Checkout hospedado + confirmación por API, verificado de punta a punta con pago real en sandbox. Nota de desarrollo: Wompi bloquea `redirect-url` apuntando a `localhost` (403 CloudFront) — hace falta un túnel público (ngrok) para probar el checkout completo en local.

Fase 4 — Reportes y comisiones

Tres secciones ancladas siempre a `Appointment.startsAt`: conteo de citas por estado; ingresos por proveedor+moneda (Stripe USD y Wompi COP **nunca** se suman entre sí); comisión por profesional calculada sobre `Service.price` de citas `COMPLETED` (nunca sobre `Payment.amount`, para funcionar igual en negocios que cobran en efectivo). Acceso restringido a OWNER/ADMIN.

Fase 5 — Multi-sede avanzado, CRM predictivo e Inventario

Gestión de sedes: modelo `ProfessionalLocation`, CRUD de sedes (OWNER únicamente), selector de sede real tanto en la agenda interna como en el booking público cuando el tenant tiene 2+ sedes.

CRM predictivo de recompra: un cliente es candidato a aviso si tuvo al menos una cita `COMPLETED`, no tiene ninguna cita futura, y pasaron 60+ días desde su última cita completada. Dos crons nuevos: `/api/cron/detect-inactive-clients` y `/api/cron/send-followup-reminders`.

Inventario: InventoryItem (catálogo por tenant), InventoryStock (saldo por ítem+sede), InventoryMovement (historial). Descuento manual en esta fase (el descuento automático por cita se agregó después, ver más abajo). Alertas de stock bajo solo visuales.

Fase 6 — Feature-gating por plan

src/lib/planLimits.ts centraliza los límites por plan (PLAN_LIMITS, getPlanLimits, planIncludesModule, hasReachedLocationLimit, hasReachedProfessionalLimit). Tope de sedes y de profesionales activos aplicado en las acciones de creación/edición, nunca solo en la UI. Gating de módulos (Inventario, Reportes, CRM predictivo) vía requireInventoryAccess / requireReportsAccess en src/lib/auth-guards.ts, con redirect a /plan-required. Bloqueo duro por PAST_DUE/CANCELLED centralizado en requireDashboardAccess → /account-locked.

Gestión de profesionales

Módulo nuevo (src/app/dashboard/[tenantSlug]/professionals/) con checklist de servicios y sedes asignadas. Tope de profesionales activos por plan (INDIVIDUAL: 1, BASICO: 3, PREMIUM: 8, PRO: sin límite) — nunca bloquea dejar o crear un profesional **inactivo**.

Equipo (invitar usuarios a una sede)

Sin infraestructura de email todavía en ese momento, así que el alta es directa: el OWNER/ADMIN crea el usuario con una contraseña temporal generada por el sistema, mostrada **una sola vez** vía cookie httpOnly. Rol independiente por sede. Reglas anti-escalación: un ADMIN no puede asignar rol OWNER ni editar a un usuario que ya es OWNER en el tenant.

Cobro automático del SaaS — Stripe Subscriptions

Tenant.stripeCustomerId/stripeSubscriptionId, checkout de suscripción y Billing Portal. Webhook de Stripe extendido con rama de suscripción (checkout.session.completed en modo subscription, invoice.paid, invoice.payment_failed, customer.subscription.deleted) sin tocar la rama existente de pagos de citas. **Verificado en vivo** con Stripe test mode real: checkout completo, cancelación real de suscripción → tenant pasa a CANCELLED → hard-lock → botón “Actualizar método de pago” funcionando.

Nota operativa importante: durante la verificación de esta fase, un comando de Prisma con --shadow-database-url mal apuntado vació accidentalmente todas las filas de la base de datos real de Neon. Se recuperó con un point-in-time restore de Neon. Lección: --shadow-database-url nunca debe apuntar a una base de datos real.

Self-service signup para tenants nuevos

Ruta pública /signup con formulario de negocio + owner + primera sede. Crea Tenant en estado TRIAL (nunca ACTIVE automáticamente), con slug autogenerated y sufijo

numérico si hay colisión. Email único a nivel global (User.email). Verificado en vivo con dos signups reales y limpieza posterior de los tenants de prueba.

Cambio de plan self-service (upgrade/downgrade)

billing/actions.ts → changeSubscriptionPlanAction, aplica el cambio de inmediato vía stripe.subscriptions.update con prorrateo. La fuente de verdad del plan es el webhook (customer.subscription.updated), nunca una actualización optimista. Un downgrade que deja al tenant por encima del límite del plan nuevo **se permite igual** (decisión de producto explícita): solo se muestra una advertencia informativa, el enforcement existente sigue bloqueando crear más recursos.

Cobro recurrente vía Wompi (Colombia)

Modelo nuevo TenantWompiCharge, separado de Payment/Appointment. Reintentos automáticos (hasta 3, a +2/+4/+7 días) con PAST_DUE en el medio y CANCELLED si se agotan. Exclusión mutua verificada con Stripe en ambos sentidos. Verificado en vivo con Wompi sandbox real: tokenización → cobro → webhook → aprobado/rechazado → reintentos → cancelación.

Recuperación y cambio de contraseña

Flujo self-service por email con link de recuperación (dependencia nueva: resend). Token de un solo uso, hash SHA-256 (nunca se guarda el token crudo), expira en 60 minutos, cooldown de 2 minutos entre solicitudes. Cambio de contraseña logueado desde /dashboard/[tenantSlug]/account. Complemento: reseteo asistido por OWNER/ADMIN desde Equipo.

Pendiente real: no hay RESEND_API_KEY configurada — el envío de email nunca se probó contra Resend de verdad (falla silenciosamente, tal como está diseñado).

Vínculo Profesional↔Usuario (login propio para un profesional)

Professional.userId ya existía sin usar en el schema — cero migraciones nuevas. Tres acciones: invitar como usuario nuevo, vincular a un usuario del equipo ya existente (preservando su rol si es OWNER), y desvincular. Alcance elegido deliberadamente: un profesional con login propio ve **exactamente lo mismo que un STAFF** (toda la agenda de su sede) — restringir a “solo mi agenda” quedó fuera, sin empezar.

Gestión de servicios

CRUD de Service (nombre, duración, precio, activo). requireServicesManageAccess sin gating de plan. Service.price sigue sin moneda propia — se interpreta como USD para Stripe y se convierte con un tipo de cambio mockeado para Wompi/COP.

Constructor de campos personalizados por tenant

TenantClientField (key/label/type/options/order/active), **aditivo** a la plantilla fija por vertical — nunca la reemplaza. key autogenerada del label y fija para siempre; type tampoco editable una vez creado. Detección de colisión de key real contra los campos fijos verificada en vivo.

Exportar reportes (CSV, PDF, gráficos)

Refactor de computeReportData como única fuente de verdad para la página, el export CSV y el export PDF (nunca pueden mostrar números distintos entre sí). Gráficos con recharts (barras por estado, por proveedor+moneda, por profesional).

Desviación arquitectónica documentada: el export PDF vive en **Pages Router** (src/pages/api/reports/[tenantSlug]/pdf.tsx), no en App Router, porque @react-pdf/renderer choca con el grafo de módulos “react-server” de Next 15 bajo src/app/. Los guards de autorización se reimplementaron a mano ahí, verificados línea por línea contra el original.

Vínculo servicio↔insumo con descuento automático de stock

Debate explícito con tres agentes especializados antes de escribir el spec. Decisión ganadora: **la cita siempre se completa**, sin importar el stock; el stock puede quedar en negativo como señal honesta de un faltante real (nunca se bloquea la cita ni se topa el descuento en 0). ServiceInventoryItem (tenant-wide), descuento atómico dentro de la misma transacción que completa la cita, solo si el plan del tenant incluye el módulo inventory.

3. Integración de WhatsApp Business (Meta) — trabajo de esta sesión

Este bloque **todavía no está reflejado en CLAUDE.md** porque se hizo en esta conversación de Cowork (no en Claude Code/VS Code). Es importante copiarlo al CLAUDE.md del proyecto o mencionarlo explícitamente al retomar.

Lo que se completó

1. **System User de Meta Business** — “PlataformaAgenda API” (ID 61592688632182, rol Admin), con activos asignados: la app “Plataforma Agenda” (permiso Desarrollar app) y la “Test WhatsApp Business Account” (permisos de plantillas y números de teléfono).
2. **Token de acceso permanente** (nunca expira) generado para ese System User, con los scopes `whatsapp_business_management` y `whatsapp_business_messaging`. Cargado en `.env` como `WHATSAPP_CLOUD_API_TOKEN`. > Nota: el primer token generado se perdió (se sobrescribió del > portapapeles antes de guardarlo). Se generó un **segundo token > permanente** con los mismos scopes, que es el que quedó cargado. El > primer token sigue técnicamente activo del lado de Meta (no se revocó) > — si en algún momento se quiere limpiar, se revoca desde Meta Business > Suite → Usuarios del sistema → “PlataformaAgenda API” → “Revocar > tokens”.
3. **Phone Number ID de prueba**: 1233870256475019 (número de prueba de Meta, +1 555 198-6649), cargado en `.env` como `WHATSAPP_PHONE_NUMBER_ID`. WhatsApp Business Account ID: 1780554303114522.
4. **Dos plantillas de mensaje creadas en WhatsApp Manager y enviadas a revisión** (estado: “En revisión” al cierre de la sesión — pendiente de aprobación por Meta):
 - `recordatorio_cita` — categoría *Utilidad*, idioma Spanish (MEX), 3 variables posicionales (nombre del cliente, servicio, fecha/hora). Cuerpo: “*Hola {{1}}, te recordamos tu cita de {{2}} el {{3}}. Si necesitas reagendar o cancelar, contáctanos. Te esperamos.*”
 - `seguimiento_recompra` — categoría *Marketing* (clasificación esperada por el tipo de mensaje — implica costo más alto y revisión más estricta que Utilidad), idioma Spanish (MEX), 2 variables (nombre del cliente, nombre del negocio). Cuerpo: “*Hola {{1}}, notamos que hace tiempo no nos visitas. En {{2}} te esperamos con gusto para tu próxima consulta.*”

Estos nombres y cantidad de variables coinciden exactamente con lo que ya espera el código (`buildReminderTemplatePayload` y `buildFollowUpTemplatePayload` en `src/lib/whatsapp.ts`) — no hace falta ningún cambio de código cuando se aprueben.

5. **Prueba de envío real exitosa**: usando el panel de pruebas de Meta for Developers (App → Casos de uso → Conectar en WhatsApp → Paso 1. Pruébalo) se envió un mensaje de plantilla pre-aprobada (“Confirmación del pedido”, la

única disponible porque las dos propias siguen en revisión) al número verificado +57 312 368 1984. El webhook de prueba confirmó "status": "delivered" — validando que el token permanente y el Phone Number ID cargados en .env funcionan de punta a punta contra la API real de Meta.

Qué falta para que WhatsApp funcione en producción

- Esperar la **aprobación de Meta** de recordatorio_cita y seguimiento_recompra (normalmente horas, a veces hasta 24-48h).
- Conseguir un **número de WhatsApp Business real** (no el número de prueba) — Jonta indicó que por ahora no puede avanzar este paso.
- Completar la **verificación de negocio** en Meta Business Suite (puede tardar de 1 a 30 días).
- Una vez exista el número de producción, **reasignar el token permanente** (o generar uno nuevo) con acceso al WABA real, no solo al de prueba.
- Confirmar en .env de producción los mismos cuatro valores: WHATSAPP_CLOUD_API_TOKEN, WHATSAPP_PHONE_NUMBER_ID, WHATSAPP_REMINDER_TEMPLATE_NAME/LANG, WHATSAPP_FOLLOWUP_TEMPLATE_N (estos dos últimos ya están correctos: recordatorio_cita/es_MX y seguimiento_recompra/es_MX).

4. Estado de las variables de entorno (.env)

No se incluyen valores reales en este documento por seguridad — solo el estado de cada una.

Variable	Estado
DATABASE_URL	Configurada (Neon)
AUTH_SECRET	Placeholder ("changeme-generar-un-valor-aleatorio-largo") — reemplazar antes de producción
STRIPE_SECRET_KEY	Configurada (modo prueba)
STRIPE_WEBHOOK_SECRET	Configurada
STRIPE_PRICE_INDIVIDUAL/BASICO/PREMIUM/PRO	Configuradas (Price IDs de prueba)
WOMPI_PUBLIC_KEY / WOMPI_PRIVATE_KEY	Configuradas (sandbox)
WOMPI_EVENTS_SECRET /	Configuradas (sandbox)
WOMPI_INTEGRITY_SECRET	
WOMPI_PRICE_COP_INDIVIDUAL/BASICO/PREMIUM/PRO	Varías — pendiente completar con montos reales según tasa de cambio
WHATSAPP_CLOUD_API_TOKEN	Configurada (token permanente, cuenta de prueba)
WHATSAPP_PHONE_NUMBER_ID	Configurada (número de prueba)
WHATSAPP_REMINDER_TEMPLATE_NAME/LANG	recordatorio_cita / es_MX
WHATSAPP_FOLLOWUP_TEMPLATE_NAME/LANG	seguimiento_recompra / es_MX
CRON_SECRET	Configurada
RESEND_API_KEY	Vacía — el envío de email falla silenciosamente
EMAIL_FROM	Configurada (sender de sandbox onboarding@resend.dev)
NEXT_PUBLIC_APP_URL	http://localhost:3000 (ajustar en producción)

5. Qué falta / pendiente para continuar

Consolidado de todos los “fuera de esta fase” mencionados en CLAUDE.md, agrupado por área.

WhatsApp

- Aprobación de las dos plantillas por Meta.
- Número de WhatsApp Business real (no el de prueba).
- Verificación de negocio en Meta Business Suite.
- Reasignar el token al WABA de producción.

Facturación / planes

- WOMPI_PRICE_COP_* sin completar con montos reales.
- Cambio de plan self-service **solo existe para Stripe** — no para tenants en Wompi.
- No hay reactivación de cuenta después de CANCELLED (ni Stripe ni Wompi) más allá del botón “Actualizar método de pago”.
- Sin actualizar tarjeta en Wompi (solo configurar por primera vez y cancelar) — no hay 3DS/3RI ni Credential On File.
- subscriptionItemId (Stripe) asume que la suscripción siempre tiene exactamente un ítem — no validado si algún día deja de ser así.
- Falta test unitario para getPlanFromStripePriceId.
- vercel.json conecta los crons pero no se confirmó si el plan de Vercel del proyecto permite la cadencia horaria prevista para send-reminders (no hay proyecto de Vercel vinculado todavía) — puede requerir ajustar a cadencia diaria si termina siendo plan Hobby.

Seguridad / cuentas

- AUTH_SECRET sigue siendo un placeholder — generar un valor real antes de producción.
- RESEND_API_KEY sin configurar — el reseteo de contraseña por email nunca se probó de verdad.
- Cambiar la contraseña **no cierra sesiones JWT activas** en otros dispositivos (limitación conocida de Auth.js con session: { strategy: "jwt" }).
- No hay rate-limiting real (CAPTCHA/IP) más allá del cooldown de 2 minutos por usuario en reset de contraseña.

Profesionales / equipo

- Login PROFESSIONAL ve exactamente lo mismo que STAFF (toda la agenda de su sede) — restringir a “solo mi agenda” es una fase completa sin empezar.
- El acceso otorgado (StaffLocationRole) al vincular un profesional **no se resincroniza automáticamente** si después cambian sus sedes asignadas — se ajusta a mano desde Equipo.
- No hay borrado real de profesionales/sedes/usuarios, solo desactivación.

Servicios / inventario

- Sin selector de moneda por Service — sigue interpretándose como USD con conversión mockeada para COP.
- Sin campo de costo/moneda en InventoryItem ni ServiceInventoryItem.
- Sin desvinculación automática de ServiceInventoryItem cuando el InventoryItem asociado se desactiva.
- Sin reporte de consumo de insumos.
- Sin categorías/proveedores de inventario.

Reportes

- Gráficos solo en la página web, no dentro del PDF exportado (decisión deliberada: requeriría un navegador headless).
- Sin exportar a otros formatos (Excel, etc.).

General / UX

- Sin constructor visual para reordenar u ocultar los campos **fijos** de la plantilla de vertical (el constructor de campos personalizados es aditivo, no reemplaza la plantilla fija).
- Sin gestión de Service por sede (sigue siendo tenant-wide).

6. Comandos útiles

```
npm install
cp .env.example .env          # y completar DATABASE_URL, etc.
docker compose up -d          # levanta Postgres local
npm run db:migrate             # crea las tablas a partir de prisma/schema.prisma
npm run db:seed                # carga datos de ejemplo (tenant, sede, servicio, cliente demo)
npm run dev                    # http://localhost:3000
```

Para probar webhooks de Stripe en local:

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

Para probar el checkout de Wompi en local (bloquea redirect-url con localhost):
levantar un túnel público (ngrok) y entrar por esa URL.

7. Archivos de referencia del proyecto

- CLAUDE.md — memoria persistente del proyecto, se lee automáticamente al abrir en Claude Code / VS Code. **Es la fuente de verdad más detallada.**
- guia-aprobacion-whatsapp-business-meta.md — guía paso a paso del trámite de WhatsApp Business en Meta (System User, token permanente, plantillas, verificación de negocio).
- Varios prompt-*.md en la raíz del repo — specs detalladas usadas para cada fase grande (cobro automático, self-service signup, cambio de plan, Wompi recurrente, reset de contraseña, vínculo profesional-usuario, gestión de servicios, constructor de campos, exportar reportes, vínculo servicio-insumo). Útiles como referencia de las decisiones tomadas en cada una.

Cómo retomar en otra conversación: pega o adjunta este PDF (o el CLAUDE.md del repo, que tiene el mismo contenido salvo la sección de WhatsApp de esta sesión) y decíle a Claude en qué fase/pendiente querés seguir. Si vas a trabajar directo en el código, referenciá la fase exacta para que no se adelante a construir cosas de fases posteriores sin necesidad.